

Projeto Integrador de Tecnologia da Informação III

Prof. Dr. Hudson Silva Borges

Módulo 3 – Boas práticas em serviços web

Unidade 2 - Documentação e versionamento de serviços

Documentação de APIs

- A economia global é multi-plataforma e **conectada por APIs.**
- Plataformas são **produtos que podem ser estendidos** e usados por terceiros para benefício próprio.
- Desenvolvedores buscam entender como usar APIs de forma efetiva, e nesse cenário, **a documentação se torna um ponto crítico.**

O que é?

- Documentação de APIs são **documentos técnicos** criados e distribuídos pela equipe de desenvolvimento.
- **Contêm instruções** de como usar efetivamente e integrar com outras APIs.
- **Tratam-se de manuais** concisos com toda informação necessária para trabalhar com a API.

O que é?

- Documentações devem detalhar:
 - Funções/serviços disponíveis;
 - Estrutura de dados usadas;
 - Argumentos necessários;
 - Respostas esperadas;
 - etc.

Por que documentar APIs?

- Documentação **impacta diretamente no uso e na adoção** de APIs
- Por que devemos documentar APIs?
 - Melhoria da **adoção** por usuários;
 - Maior **conscientização**;
 - **Redução** do tempo de suporte e custos;
 - **Melhoria** da manutenção.

OpenAPI e Swagger

Especificação para documentação de *web services*

Especificação da OpenAPI

- **OpenAPI Specification** é um formato para especificação de APIs REST que pode ser escrita em YAML ou JSON.

The screenshot displays the Swagger Editor interface. On the left, a dark-themed code editor shows a YAML specification for the Swagger Petstore API. The specification includes the OpenAPI version (3.0.0), API title (Swagger Petstore), license (MIT), and a list of endpoints. The first endpoint is a GET request to /pets, which lists all pets and includes a query parameter 'limit' to specify the number of items to return. The right panel shows a light-themed preview of the API documentation, titled 'Swagger Petstore 1.0.0 OAS3'. It features a 'Servers' section with a dropdown menu set to 'http://petstore.swagger.io/v1'. Below this, the 'pets' section is expanded, showing three endpoints: a GET request to /pets (List all pets), a POST request to /pets (Create a pet), and a GET request to /pets/{petId} (Info for a specific pet). The bottom of the interface shows a 'Models' section.

```
1 openapi: "3.0.0"
2 info:
3   version: 1.0.0
4   title: Swagger Petstore
5   license:
6     name: MIT
7 servers:
8   - url: http://petstore.swagger.io/v1
9 paths:
10  /pets:
11    get:
12      summary: List all pets
13      operationId: listPets
14      tags:
15        - pets
16      parameters:
17        - name: limit
18          in: query
19          description: How many items to return at one
20            time (max 100)
21          required: false
22          schema:
23            type: integer
24            format: int32
25      responses:
26        '200':
27          description: A paged array of pets
28          headers:
29            x-next:
30              description: A link to the next page of
31                responses
32              schema:
33                type: string
34              content:
```

Fonte:

<https://link.ufms.br/AI4ZB>

Estrutura básica

```
1  openapi: 3.0.0
2  info:
3    title: Sample API
4    description: Optional multiline or single-line description in
5    [CommonMark](http://commonmark.org/help/) or HTML.
6    version: 0.1.9
7  servers:
8    - url: http://api.example.com/v1
9      description: Optional server description, e.g. Main (production) server
10   - url: http://staging-api.example.com
11     description: Optional server description, e.g. Internal staging server
    for testing
```

← Metadados

← Servidores

Fonte:

<https://link.ufms.br/AI4ZB>

Estrutura básica

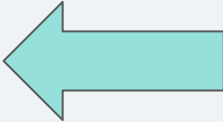
```
1  paths:
2    /users:
3      get:
4        summary: Returns a list of users.
5        description: Optional extended description in CommonMark or HTML
6        responses:
7          "200":
8            description: A JSON array of user names
9            content:
10              application/json:
11                schema:
12                  type: array
13                  items:
14                    type: string
```

← Caminhos

Fonte: <https://link.ufms.br/AI4ZB>

Estrutura básica

```
1  paths:
2    /users:
3      get:
4        parameters:
5          - in: query
6            name: role
7            schema:
8              type: string
9              enum: [user, poweruser, admin]
10         required: true
```

 **Parâmetros**
(GET /users?role=admin)

Fonte:

<https://link.ufms.br/AI4ZB>

- **Swagger** é um conjunto de ferramentas open-source que implementa as especificações do OpenAPI
- Editor online:
 - <https://editor.swagger.io>
- Especificação:
 - <https://swagger.io/specification>

Documentações integradas em código

- Manter a documentação atualizada por ser **desafiador** e, em muitos casos, desenvolvedores acabam sendo negligentes.
- Processos integrados podem facilitar a atualização, como defendido pela filosofia “*documentation as code*”.
- Nesse modelo, buscamos integrar documentação ao desenvolvimento **usando as mesmas ferramentas**.

Documentações integradas em código

- Atualmente, existem diversas bibliotecas que possibilitam a escrita de documentos no formato OpenAPI a partir de **anotações em código fonte**.

```
@RestController
@RequestMapping("/filmes")
@Tag(name = "Filmes", description = "API de filmes")
public class FilmeController {
```

Exemplo de documentação usando
SpringDoc (springdoc.org)

```
@GetMapping
@Operation(summary = "Lista todos os filmes", description = "Lista todos os fi
@ApiResponse(responseCode = "200", description = "Lista de filmes")
public Collection<FilmeDTO> getAllFiles(
```

Fonte: Elaboração própria

Documentações integradas em código

- Atualmente, existem diversas bibliotecas que possibilitam a redação de documentos no formato OpenAPI a partir de **anotações em código fonte**.
- Diversos *frameworks* populares, como *NestJS* (JavaScript), *FastAPI* (Python) e *SpringBoot* (Java) incluem pacotes para geração automática da documentação, bastando adicionar as configurações.

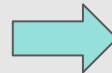
Versionamento de serviços

Criando APIs retrocompatíveis

Importância de **compatibilidade**

- Serviços se modificam e novas demandas surgem.
- Serviços estabelecem **contratos** de uso com seus clientes; mas e se precisamos alterá-los?

```
{  
  "passengerId": 1,  
  "destination": "Avenida Faria Lima, 1300",  
}
```



```
{  
  "passengerId": 1,  
  "destination": "Avenida Faria Lima, 1300",  
  "origin": "Avenida Paulista, 100"  
}
```

Fonte: (Saudate, 2021) adaptado

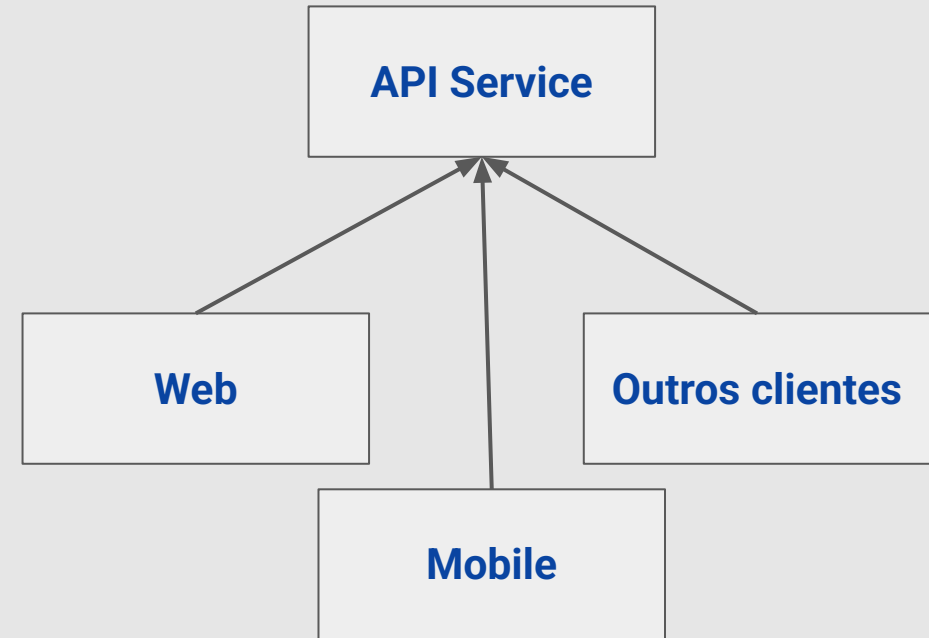
Importância de compatibilidade

```
{  
  "passengerId": 1,  
  "destination": "Avenida Faria Lima, 1300",  
  "origin": "Avenida Paulista, 100"  
}
```



```
{  
  "passengerId": 1,  
  "destination": {  
    "name": "Avenida Faria Lima, 1300",  
    "zipCode": "05426-100"  
  },  
  "origin": "Avenida Paulista, 100"  
}
```

Fonte: Saudate, 2021.



Todos estão usando
a mesma versão?

Como versionar APIs?

- Se você enfrenta essas situações, existem algumas abordagens:
 - Modificar a URL para incluir a versão;
 - Informar a versão na requisição:
 - Via parâmetros;
 - Via *header*.

Versionamento na URL

- Deixa evidente a versão utilizada.
- Fácil de criar regras de roteamento.
- **Exemplos:**

<https://v1.service.facom.ufms.br/endpoint>

<https://service.facom.ufms.br/v1/endpoint>

Versionamento via **parâmetros**

- Fácil, mas pouco escalável;
- Difícil roteamento e pouco efetivo para os diferentes métodos;
- **Exemplos:**

<https://service.facom.ufms.br/endpoint?v=1>

Versionamento via **cabeçalho**

- Transferência por meio de metadados;
- Modelo de negociação flexível.
- **Exemplos:**

Accept-Version: **v1**;q=0.8,v2;q=0.6

Accept: application/vnd.service+json; **version=1.0**

Conclusões

- Documentação de API permite maior disseminação e adoção na distribuição de serviços globais.
- Para times de desenvolvimento, documentações deixam APIs mais consistentes e melhoram a comunicação da equipe.
- O versionamento adequado de APIs é crítico em ambientes onde existem múltiplos usuários com diferentes clientes/versões.

Atividade de *checkout*

- Você deverá mapear, propor e disponibilizar uma estratégia de documentação para a aplicação projetada e desenvolvida no âmbito da disciplina Projeto Integrador.
- Além de documentos técnicos para serviços internos, considere uma proposta de documentação de jornadas de usuário da aplicação.

Referências

NETO, Tarcizio Vieira. **Modelo de processo para desenvolvimento de aplicações seguras.**

Disponível em: <https://link.ufms.br/nsb26> . Acesso em: 22 nov. 2024.

OPENAPI INITIATIVE. **Getting started:** OpenAPI documentation. Disponível em:

<https://link.ufms.br/yiSJI> Acesso em: 22 nov. 2024.

SAUDATE, Alexandre. **APIs REST:** seus serviços prontos para o mundo real. São Paulo: Casa do Código, 2021.

Licenciamento



Respeitadas as formas de citação formal de autores de acordo com as normas da ABNT NBR 6023 (2018), a não ser que esteja indicado de outra forma, todo material desta apresentação está licenciado sob uma [Licença Creative Commons - Atribuição 4.0 Internacional](https://creativecommons.org/licenses/by/4.0/).

